

# SESAP — Stanbic IBTC Enterprise Support & Automation Platform

UiPath Developer Challenge · End-to-End Orchestration (Coded App + RPA + Maestro) · Bank Developer Edition

- Coded App
- Data Fabric
- Maestro BPMN
- RPA Robot
- AI Agent (Claude)
- Live & verified

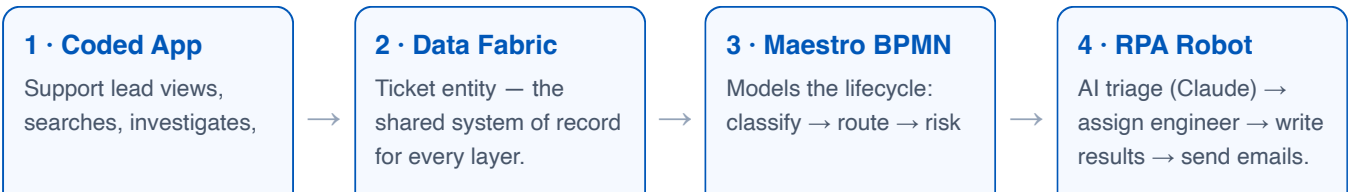
## 1. Executive summary

SESAP is a support-ticket operations platform that spans the full UiPath stack. A support lead works in a **Coded App** that reads and writes ticket data in **UiPath Data Fabric**. When a ticket is created, the app sets a real automated process in motion: a **Maestro (BPMN)** process orchestrates the ticket lifecycle and invokes an **RPA robot**, which runs an **AI triage agent** (Anthropic Claude), assigns the ticket, and writes the results back to Data Fabric — which the app then reflects. Every stage sends branded email notifications with a one-click deep link back to the ticket.

**The full loop is deployed and verified live** — an action in the app flows through Maestro and the robot to a visible data update, end to end.

|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Live application | <code>https://stanbgdgzsbd.uipath.host/sesap-support-platform</code>                                                |
| Environment      | UiPath Automation Cloud · org <code>stanbgdgzsbd</code> · tenant <code>DefaultTenant</code>                         |
| Front end        | UiPath Coded App (React + Vite), official <code>@uipath/uipath-typescript</code> SDK                                |
| System of record | Data Fabric <code>Ticket</code> entity (24 fields)                                                                  |
| Orchestration    | Maestro process <code>TicketLifecycle</code> → RPA process <code>TicketAutomation</code> (folder Shared/SESAP-Live) |
| AI               | Anthropic Claude ( <code>claude-haiku-4-5</code> ) called by the robot inside the flow                              |

## 2. Reference architecture



creates tickets & starts the orchestration.

gateway → assign → invoke robot.

↑ The robot writes back to **Data Fabric**, and the **Coded App** re-reads it — closing the loop. Email notifications fire at each lifecycle stage.

## The end-to-end loop, step by step

1. A user submits a ticket in the Coded App.
2. The app writes it to the Data Fabric `Ticket` entity with `AssignmentStatus = "Unassigned"`.
3. The app starts the **Maestro** `TicketLifecycle` process (a job started via the SDK — the "UI action → orchestration" trigger).
4. Maestro runs the lifecycle — **Classify** → **Route** → **risk gateway (auto vs. manual)** → **Assign** — then its service task invokes the RPA robot and waits for it.
5. The robot calls the **AI agent** (Claude) to categorise, prioritise, summarise and recommend; assigns an engineer by category; and `UpdateEntityRecord` writes assignee, status, AI category/priority/summary/recommendation to Data Fabric.
6. The robot sends branded emails (requester + assignee) with an **"Open ticket in SESAP"** deep link.
7. The app re-reads Data Fabric and shows the ticket now **Assigned**, with the AI triage panel populated.

**Verified live.** A ticket created as *Unassigned* with only a free-text description ("app logs me out during a transfer") was auto-triaged by the AI to **Category: Digital Banking, Priority: urgent**, routed to **Tier 2 Support** → **Kunle Adeyemi**, written back to Data Fabric as **Assigned**, and surfaced in the app — with both notification emails delivered. The loop runs from a single click.

## 3. The four layers in detail

### 3.1 Coded App (operational front end)

- UiPath Coded App (React/Vite) using the official UiPath TypeScript SDK; auth via the platform-injected context.
- **Lists all tickets live from Data Fabric** with search, multi-column sort, and filters.
- **Ticket detail** view — full record, SLA countdown, activity trail, attachments, and the **AI Triage panel**.
- **Create ticket** that persists to Data Fabric and starts the orchestration.
- **Role-based portals** — distinct Staff vs. Customer experiences.
- **Live analytics** — KPIs, created-vs-resolved trend, status/priority/department distributions, SLA compliance, and robot health computed from real Orchestrator job outcomes.
- **Deep link** — opens a specific ticket from an email link ( `?ticket=<Ref>` ), surviving the sign-in round-trip.

### 3.2 Data Fabric (system of record)

Entity `Ticket` — 19 business fields + 5 system fields. Key fields:

| Field                                           | Type             | Role                                                 |
|-------------------------------------------------|------------------|------------------------------------------------------|
| <code>Ref</code>                                | Text (unique)    | Human reference, e.g. SESAP-1042                     |
| <code>Subject</code> / <code>Description</code> | Text / Multiline | Ticket content (input to the AI)                     |
| <code>Category</code> / <code>Priority</code>   | Text             | Set by the AI agent; drive routing                   |
| <code>AssignedTo</code>                         | Text             | Engineer, written by the robot                       |
| <code>AssignmentStatus</code>                   | Text             | <b>Write-back proof field: Unassigned → Assigned</b> |
| <code>SubCategory</code> / <code>Tags</code>    | Text             | AI summary / AI recommendation                       |
| <code>RequesterEmail</code>                     | Text             | Notification recipient                               |

### 3.3 Maestro (orchestration brain)

- BPMN process `TicketLifecycle` with a start event, four tasks, exclusive gateways, and end events.
- Models the lifecycle: **Ticket Submitted** → **Classify** → **Route to Team** → **risk gateway (auto-assign vs. manual)** → **Assign Ticket** → **Run Auto-Assign Robot**.
- The decision gateway routes high-risk tickets to a manual path and everything else to automated assignment.
- Invokes the RPA robot via an `Orchestrator.StartJob` service task (bound to the published RPA release) — so state is shared across the solution through the robot's Data Fabric writes.

### 3.4 RPA workflow (the hands)

- Published process `TicketAutomation` using real UiPath Data Service activities — `QueryEntityRecords` and `UpdateEntityRecord` against the `Ticket` entity.
- For each unassigned ticket: run AI triage → set category/priority → route to team → assign engineer → write back to Data Fabric → send lifecycle emails.
- Invoked by Maestro (batch mode) and by the app for lifecycle emails (argument-driven mode) — one release, two modes.

## 4. AI triage agent

The standout capability: a genuine AI agent invoked **inside the orchestration**. When the robot processes a ticket it calls Anthropic Claude ( `claude-haiku-4-5` ) with the ticket's subject and description and receives structured JSON:

- **Category** — one of the bank's support categories (drives routing/assignment).
- **Priority** — low / medium / high / urgent.
- **Summary** — a one-line synopsis for the support lead.
- **Recommendation** — a suggested resolution / next action.

The agent's output is written back to Data Fabric and shown in the app's **AI Triage panel**. The API key is stored securely as an Orchestrator asset ( `SESAP_Anthropic_Key` ) — never in code. This also fulfils the "enrich & act" stage of the ticket lifecycle.

## 5. Email notifications (full lifecycle)

Real emails via the UiPath Gmail Integration Service connection — branded HTML cards with an **"Open ticket in SESAP"** deep-link button, delivered at each stage:

| Lifecycle stage                  | Recipients           |
|----------------------------------|----------------------|
| Ticket created (acknowledgement) | Requester            |
| Assigned                         | Requester + Assignee |
| Escalated                        | Requester + Assignee |
| Resolved / Closed                | Requester            |

## 6. Baseline requirements — compliance

| Requirement                                                                | Status                 |
|----------------------------------------------------------------------------|------------------------|
| Coded App connected to the environment                                     | Met                    |
| Lists all tickets live from Data Fabric; search & sort                     | Met                    |
| Ticket detail view                                                         | Met                    |
| Create ticket persists to Data Fabric                                      | Met                    |
| In-app action starts the orchestration                                     | Met                    |
| Ticket entity with required fields + a status field the automation updates | Met (AssignmentStatus) |

|                                                                       |                     |
|-----------------------------------------------------------------------|---------------------|
| Published RPA reads/writes Data Fabric, invoked by the orchestration  | Met                 |
| BPMN: start + ≥3 tasks + gateway + end; models lifecycle; invokes RPA | Met                 |
| Full loop works & is demonstrable                                     | Met (verified live) |

## 7. Bonus features delivered

| Bonus            | What was delivered                                                                                                                          |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Agentic / AI     | Claude triage agent (categorise + prioritise + summarise + recommend) invoked within the flow; results drive routing and appear in the app. |
| Role-based views | Distinct Staff and Customer portals.                                                                                                        |
| Analytics / KPIs | In-app dashboard from live data: open by priority, status mix, SLA compliance, trend, robot health.                                         |
| Notifications    | Real Gmail notifications across the full lifecycle, to both parties, with deep links.                                                       |
| Polished UX      | Enterprise, on-brand interface with light/dark themes.                                                                                      |

## 8. Deployed resources

|                    |                                                                       |
|--------------------|-----------------------------------------------------------------------|
| Coded App          | <code>https://stanbgdgzsb.d.uipath.host/sesap-support-platform</code> |
| Data Fabric entity | <code>Ticket</code>                                                   |
| Maestro process    | <code>TicketLifecycle</code> (Shared/SESAP-Live)                      |
| RPA process        | <code>TicketAutomation</code> (Shared/SESAP-Live)                     |
| AI key (secure)    | Orchestrator asset <code>SESAP_Anthropic_Key</code>                   |
| Email              | Gmail Integration Service connection                                  |

## 9. How to demonstrate

1. Open the app and sign in; show the live ticket queue (search & sort) and a ticket detail.
2. Create a ticket → watch the notifications (stored in Data Fabric → orchestration started).
3. Show the Maestro instance flowing through classify → route → assign → run robot.
4. Show the RPA job, then the ticket in Data Fabric now Assigned with AI triage populated.
5. Back in the app: the ticket is Assigned; open the AI Triage panel; open the assignee email and click "Open ticket in SESAP".

